COIMBATORE INSTITUTE OF TECHNOLOGY
Civil Aerodrome Post, Coimbatore,
Tamilnadu, India – 641014

DATA STRUCTURES AND ALGORITHMS LABORATORY
(22MDC26)

Medical Prescription Tracker

2NDSEMESTER CAT PROJECT

SUBMITTED BY:
Mithra Omprakash
240371673722027
M.Sc (DCS) – 1 YEAR

SUBMITTED TO:
Dr. V.Radhamani
Ms. S.Dharani

ABSTRACT

PROBLEM STATEMENT

Managing multiple prescriptions and medication schedules can be difficult, especially for patients undergoing long-term treatment or taking medications at different times of the day. Traditional methods like paper tracking or memory-based routines often lead to missed doses or incorrect intake. Hence, there is a need for a simple yet structured system that helps in organizing and monitoring a patient's medication records efficiently.

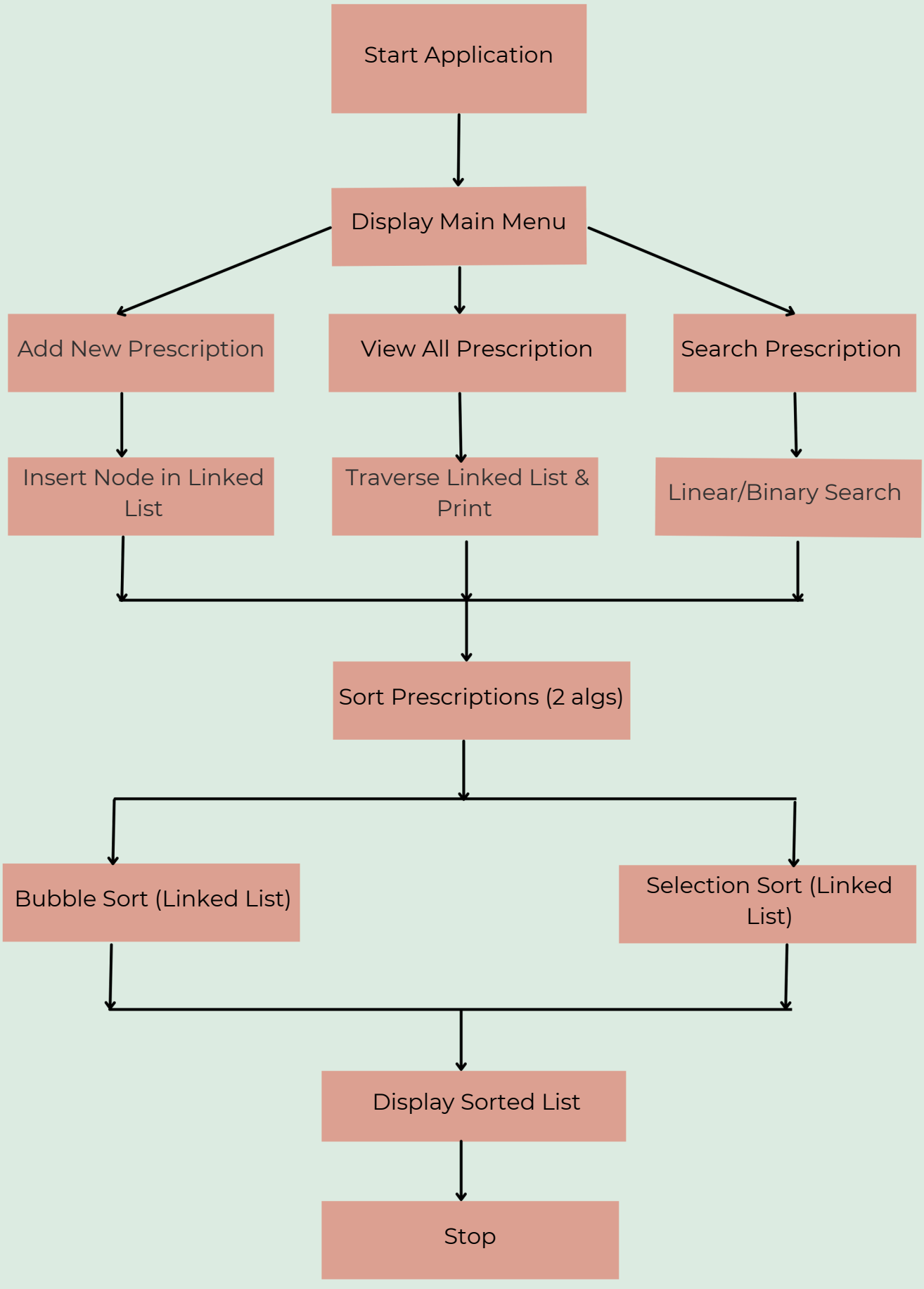
PROPOSED SOLUTION :

This project presents a Medical Prescription Tracker developed in the C programming language, utilizing Linked Lists to dynamically store and manage medication data. Each medicine is represented as a node in the list, containing relevant details such as medicine name, dosage, start and end date, and intake time (morning, afternoon, night).
The system allows users to:

- Add, update, or remove prescription records
- Display the complete list of current medications
- Search for a medicine by name using Linear Search and Binary Search (post array conversion)
- Sort medications by name or start date using Bubble Sort and Selection Sort

To demonstrate performance, the project also includes a comparative analysis of the implemented searching and sorting algorithms by measuring their execution time . This project provides a real-time, user-friendly, and extensible application of Linked Lists in healthcare data management.

SYSTEM FLOW DIAGRAM



ALGORITHM

1. START THE APPLICATION

Step 1: Initialize head pointer of the linked list to NULL.

Step 2: Display the main menu with options:

- **Add Prescription**
- **View Prescriptions**
- **Update Prescription**
- **Delete Prescription**
- **Search Prescription**
- **Sort Prescriptions**
- **Exit**

2. ADD PRESCRIPTION

Input: Medicine Name, Dosage, Start Date, End Date, Time of Intake

Output: Updated linked list with the new prescription.

Step 1: Create a new node (malloc).

Step 2: Store all input data into the new node.

Step 3: If head == NULL, make this node the head.

Step 4: Else, traverse to the last node and append the new node.

Step 5: Set newNode->next = NULL.

3. DISPLAY ALL PRESCRIPTIONS

Step 1: If head == NULL, print "No prescriptions found."

Step 2: Else, traverse the linked list:

- For each node, print medicine details:
  - Name, dosage, start & end date, intake time

ALGORITHM

4. UPDATE PRESCRIPTION

Input: Medicine Name (or unique ID, if implemented)

Step 1: Traverse the linked list to find matching medicine name.

Step 2: If found, allow user to change:

- Dosage, dates, time of intake.

Step 3: Update fields in that node.

5. DELETE PRESCRIPTION

Input: Medicine Name

Step 1: Traverse list and maintain two pointers: prev and curr.

Step 2: If curr->medName == input, then:

- If curr == head, update head = head->next
- Else, do prev->next = curr->next

Step 3: Free memory of curr.

6. SEARCH PRESCRIPTION

a) Linear Search (Linked List)

Input: Medicine Name

Step 1: Traverse linked list node by node.

Step 2: If strcmp(curr->medName, input) == 0, print and return the node.

Step 3: If not found, print "Not Found".

b) Binary Search (On Converted Array)

Input: Sorted array of medicine structs

Step 1: Use Binary Search algorithm by comparing medicine name.

Step 2: Return index or -1 if not found.

ALGORITHM

7. SORT PRESCRIPTIONS

a) Bubble Sort (Linked List)

Step 1: Repeat until no swaps:

- Compare adjacent nodes
- If node1->name > node2->name, swap data

Step 2: End when full list is sorted.

b) Selection Sort (Array Version)

Step 1: Convert Linked List to Array.

Step 2: For each index i, find min element in rest of array.

Step 3: Swap arr[i] and arr[min].

Step 4: Print sorted list or convert array back to linked list.

8. EXIT THE APPLICATION

Step 1: Free all memory (free() every node).

Step 2: Exit the program .

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
 #include <string.h>

typedef struct Prescription {
    char medName[50];
    char dosage[30];
    char startDate[20];
    char endDate[20];
    char intakeTime[20];
    struct Prescription* next;
} Prescription;

Prescription* head = NULL;

// Create a new prescription node
Prescription* createNode() {
    Prescription* newNode =
(Prescription*)malloc(sizeof(Prescription));
    printf("Enter medicine name: ");
    scanf(" %[^\n]", newNode->medName);
    printf("Enter dosage: ");
    scanf(" %[^\n]", newNode->dosage);
    printf("Enter start date (dd-mm-yyyy): ");
    scanf(" %[^\n]", newNode->startDate);
    printf("Enter end date (dd-mm-yyyy): ");
    scanf(" %[^\n]", newNode->endDate);
    printf("Enter time of intake (Morning/Night/etc): ");
    scanf(" %[^\n]", newNode->intakeTime);
    newNode->next = NULL;
    return newNode;
}
```

SOURCE CODE

```
// Add Prescription
void addPrescription() {
    Prescription* newNode = createNode();
    if (head == NULL) {
        head = newNode;
    } else {
        Prescription* temp = head;
        while (temp->next)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Prescription added successfully!\n");
}

// Display Prescriptions
void displayPrescriptions() {
    if (head == NULL) {
        printf("No prescriptions found.\n");
        return;
    }
    Prescription* temp = head;
    printf("\n--- Prescription List ---\n");
    while (temp) {
        printf("\nMedicine: %s\nDosage: %s\nStart Date: %s\nEnd
Date: %s\nIntake Time: %s\n",
               temp->medName, temp->dosage, temp->startDate,
temp->endDate, temp->intakeTime);
        temp = temp->next;
    }
}
```

SOURCE CODE

```
// Linear Search
void searchPrescriptionLinear() {
    char name[50];
    printf("Enter medicine name to search: ");
    scanf(" %[^\n]", name);
    Prescription* temp = head;
    while (temp) {
        if (strcmp(temp->medName, name) == 0) {
            printf("\nFound Prescription:\nDosage: %s\nStart Date:
%s\nEnd Date: %s\nIntake Time: %s\n",
                   temp->dosage, temp->startDate, temp->endDate,
temp->intakeTime);
            return;
        }
        temp = temp->next;
    }
    printf("Prescription not found.\n");
}

// Bubble Sort by Medicine Name
void bubbleSort() {
    if (!head) return;

    Prescription *i, *j;
    char tempMed[50], tempDosage[30], tempStart[20],
tempEnd[20], tempTime[20];

    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (strcmp(i->medName, j->medName) > 0) {
                // Swap fields
                strcpy(tempMed, i->medName);
```

SOURCE CODE

```
                strcpy(i->medName, j->medName);
                strcpy(j->medName, tempMed);
                strcpy(tempDosage, i->dosage);
                strcpy(i->dosage, j->dosage);
                strcpy(j->dosage, tempDosage);
                strcpy(tempStart, i->startDate);
                strcpy(i->startDate, j->startDate);
                strcpy(j->startDate, tempStart);
                strcpy(tempEnd, i->endDate);
                strcpy(i->endDate, j->endDate);
                strcpy(j->endDate, tempEnd);
                strcpy(tempTime, i->intakeTime);
                strcpy(i->intakeTime, j->intakeTime);
                strcpy(j->intakeTime, tempTime);
            }
        }
    }

    printf("Prescriptions sorted successfully by Bubble Sort.\n");
}
// Selection Sort by Medicine Name
void selectionSort() {
    if (!head) return;

    Prescription *i, *j, *min;
    char tempMed[50], tempDosage[30], tempStart[20],
    tempEnd[20], tempTime[20];

    for (i = head; i != NULL; i = i->next) {
        min = i;
        for (j = i->next; j != NULL; j = j->next) {
            if (strcmp(j->medName, min->medName) < 0) {
                min = j;
            }
```

SOURCE CODE

```
    }
    if (min != i) {
        // Swap fields
        strcpy(tempMed, i->medName);
        strcpy(i->medName, min->medName);
        strcpy(min->medName, tempMed);
        strcpy(tempDosage, i->dosage);
        strcpy(i->dosage, min->dosage);
        strcpy(min->dosage, tempDosage);
        strcpy(tempStart, i->startDate);
        strcpy(i->startDate, min->startDate);
        strcpy(min->startDate, tempStart);
        strcpy(tempEnd, i->endDate);
        strcpy(i->endDate, min->endDate);
        strcpy(min->endDate, tempEnd);
        strcpy(tempTime, i->intakeTime);
        strcpy(i->intakeTime, min->intakeTime);
        strcpy(min->intakeTime, tempTime);
    }
 }

    printf("Prescriptions sorted successfully by Selection Sort.\n");
}
// Convert Linked List to Array
int listToArray(Prescription* arr[]) {
    int count = 0;
    Prescription* temp = head;
    while (temp) {
        arr[count++] = temp;
        temp = temp->next;
    }
    return count;
}
```

SOURCE CODE

```
// Binary Search
void binarySearch() {
    Prescription* arr[100];
    int n = listToArray(arr);

    // First, Selection Sort on array
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i+1; j < n; j++) {
            if (strcmp(arr[j]->medName, arr[min]->medName) < 0)
                min = j;
        }
        if (min != i) {
            Prescription* temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
        }
    }

    char name[50];
    printf("Enter medicine name to search: ");
    scanf(" %[^\n]", name);

    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high)/2;
        int cmp = strcmp(arr[mid]->medName, name);
        if (cmp == 0) {
            printf("\nFound Prescription:\nDosage: %s\nStart Date:
%s\nEnd Date: %s\nIntake Time: %s\n",
                arr[mid]->dosage, arr[mid]->startDate, arr[mid]-
>endDate, arr[mid]->intakeTime);
```

SOURCE CODE

```
            return;
        } else if (cmp < 0)
            low = mid + 1;
        else
            high = mid - 1;
    }
    printf("Prescription not found.\n");
}

// Update Prescription
void updatePrescription() {
    char name[50];
    printf("Enter medicine name to update: ");
    scanf(" %[^\n]", name);

    Prescription* temp = head;
    while (temp) {
        if (strcmp(temp->medName, name) == 0) {
            printf("Enter new dosage: ");
            scanf(" %[^\n]", temp->dosage);
            printf("Enter new start date (dd-mm-yyyy): ");
            scanf(" %[^\n]", temp->startDate);
            printf("Enter new end date (dd-mm-yyyy): ");
            scanf(" %[^\n]", temp->endDate);
            printf("Enter new intake time: ");
            scanf(" %[^\n]", temp->intakeTime);
            printf("Prescription updated successfully.\n");
            return;
        }
        temp = temp->next;
    }
    printf("Prescription not found.\n");
}
```

SOURCE CODE

```
// Delete Prescription
void deletePrescription() {
    char name[50];
    printf("Enter medicine name to delete: ");
    scanf(" %[^\n]", name);

    Prescription *temp = head, *prev = NULL;
    while (temp) {
        if (strcmp(temp->medName, name) == 0) {
            if (prev == NULL) {
                head = temp->next;
            } else {
                prev->next = temp->next;
            }
            free(temp);
            printf("Prescription deleted successfully.\n");
            return;
        }
        prev = temp;
        temp = temp->next;
    }
    printf("Prescription not found.\n");
}

// Free memory
void freeList() {
    Prescription* temp;
    while (head) {
        temp = head;
        head = head->next;
        free(temp);
    }
}
```

SOURCE CODE

```
// Main Menu
int main() {
    int choice;
    clock_t start, end;

    do {
        printf("\n===== Medical Prescription Tracker =====\n");
        printf("1. Add Prescription\n");
        printf("2. Display Prescriptions\n");
        printf("3. Linear Search (Medicine Name)\n");
        printf("4. Binary Search (Medicine Name)\n");
        printf("5. Bubble Sort (by Medicine Name)\n");
        printf("6. Selection Sort (by Medicine Name)\n");
        printf("7. Update Prescription\n");
        printf("8. Delete Prescription\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: addPrescription(); break;
            case 2: displayPrescriptions(); break;
            case 3:
                start = clock();
                searchPrescriptionLinear();
                end = clock();
                printf("Time taken: %.6f sec\n", (double)(end -
start)/CLOCKS_PER_SEC);
                break;
```

SOURCE CODE

```
            case 4:
                start = clock();
                binarySearch();
                end = clock();
                printf("Time taken: %.6f sec\n", (double)(end start)/
                CLOCKS_PER_SEC);
                break;
            case 5:
                start = clock();
                bubbleSort();
                end = clock();
                printf("Time taken: %.6f sec\n", (double)(end - start)/
                CLOCKS_PER_SEC);
                break;
            case 6:
                start = clock();
                selectionSort();
                end = clock();
                printf("Time taken: %.6f sec\n", (double)(end - start)/
                CLOCKS_PER_SEC);
                break;
            case 7: updatePrescription(); break;
            case 8: deletePrescription(); break;
            case 0:
                freeList();
                printf("Goodbye!\n");
                break;
            default:
                printf("Invalid choice. Try again.\n");
        }
    } while (choice != 0);

    return 0;
}
```

SAMPLE OUTPUT

```
===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 1
Enter medicine name: Paracetamol
Enter dosage: 500 mg
Enter start date (dd-mm-yyyy): 10-4-2025
Enter end date (dd-mm-yyyy): 14-4-2025
Enter time of intake (Morning/Night/etc): morning
Prescription added successfully!

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 1
Enter medicine name: Azee
Enter dosage: 250 mg
Enter start date (dd-mm-yyyy): 10-4-2025
Enter end date (dd-mm-yyyy): 15-4-2025
Enter time of intake (Morning/Night/etc): night
Prescription added successfully!

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 1
Enter medicine name: Pan40
Enter dosage: 40 mg
Enter start date (dd-mm-yyyy): 25-4-2025
Enter end date (dd-mm-yyyy): 30-4-2025
```

SAMPLE OUTPUT

```
Enter time of intake (Morning/Night/etc): evening
Prescription added successfully!

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 2

--- Prescription List ---

Medicine: Paracetamol
Dosage: 500 mg
Start Date: 10-4-2025
End Date: 14-4-2025
Intake Time: morning

Medicine: Azee
Dosage: 250 mg
Start Date: 10-4-2025
End Date: 15-4-2025
Intake Time: night

Medicine: Pan40
Dosage: 40 mg
Start Date: 25-4-2025
End Date: 30-4-2025
Intake Time: evening

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 3
Enter medicine name to search: Pan40

Found Prescription:
Dosage: 40 mg
Start Date: 25-4-2025
End Date: 30-4-2025
```

SAMPLE OUTPUT

```
Intake Time: evening
Time taken: 5.282000 sec

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 4
Enter medicine name to search: Azee

Found Prescription:
Dosage: 250 mg
Start Date: 10-4-2025
End Date: 15-4-2025
Intake Time: night
Time taken: 4.734000 sec

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 5
Prescriptions sorted successfully by Bubble Sort.
Time taken: 0.002000 sec

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 6
Prescriptions sorted successfully by Selection Sort.
Time taken: 0.004000 sec
```

SAMPLE OUTPUT

```
===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 7
Enter medicine name to update: Pan40
Enter new dosage: 50 mg
Enter new start date (dd-mm-yyyy): 25-10-2025
Enter new end date (dd-mm-yyyy): 31-10-2025
Enter new intake time: evening
Prescription updated successfully.

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: 8
Enter medicine name to delete: Azee
Prescription deleted successfully.

===== Medical Prescription Tracker =====
1. Add Prescription
2. Display Prescriptions
3. Linear Search (Medicine Name)
4. Binary Search (Medicine Name)
5. Bubble Sort (by Medicine Name)
6. Selection Sort (by Medicine Name)
7. Update Prescription
8. Delete Prescription
0. Exit
Enter your choice: _
```

RESULT ANALYSIS

SEARCHING ALGORITHMS COMPARISON

Algorithm	Best Case	Worst Case	Time Complexity	Remarks
Linear Search	O(1)	O(n)	O(n)	Simple and works on unsorted data, inefficient for large data
Binary Search	O(1)	O(log n)	O(log n)	Very fast but works only on sorted data (array conversion needed)

SORTING ALGORITHMS COMPARISON

Algorithm	Best Case	Worst Case	Time Complexity	Space Complexity	Remarks
Bubble Sort	O(1)	$O(n^2)$	$O(n^2)$	O(1)	Simple and works on unsorted data, inefficient for large data
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Very fast but works only on sorted data (array conversion needed)

INFERENCE

- The project efficiently tracks and organizes prescriptions using Linked Lists—a dynamic and memory-efficient data structure.
- The inclusion of Linear and Binary Search provides options for both small and larger datasets.
- The use of both Bubble Sort and Selection Sort allows performance comparison, helping users understand how different algorithms behave.
- Though basic, the project is extendable, beginner-friendly, and demonstrates core data structure concepts well—making it ideal for academic submissions and résumé inclusion.

The implementation demonstrates that while simple algorithms like Linear Search and Bubble Sort are easy to understand and work well for small datasets, they are not scalable. Binary Search proves to be significantly faster on larger, sorted datasets, though it requires extra processing to convert the list into an array. Similarly, Selection Sort is consistent but inefficient for large inputs. This project showcases how different algorithms perform under various conditions and emphasizes the importance of choosing the right algorithm based on data structure and dataset size. It effectively blends theoretical knowledge with practical application, making it both educational and professionally relevant.

FUTURE ENHANCEMENTS

- User Authentication System

Implementing a basic login/signup system would allow individual users (patients or doctors) to securely access and manage their own prescription records.

- Reminder Notifications

Adding a reminder feature (via email or terminal-based alerts) to notify users when it's time to take a medication or when a prescription is about to end.

- Date-Based Filtering and Sorting

Allow sorting prescriptions based on start date or end date and filtering based on current/expired prescriptions for easier tracking.

- Export to PDF/CSV

Enable users to export their prescription history to a downloadable format like PDF or CSV for easy sharing with doctors or pharmacists.

- Doctor and Pharmacy Mapping

Including optional fields to track prescribing doctors and preferred pharmacies can help in long-term prescription management.

- Integration with Mobile App / Cloud Sync

Building a mobile version of the tracker with cloud synchronization will allow users to access their prescriptions across multiple devices.

- Advanced Data Structures

In future versions, transition from simple linked lists to more complex data structures like trees or hash maps to improve searching and categorization performance.

REFERENCES

- Websites
- GeeksforGeeks – Data Structures in C
- TutorialsPoint – Data Structures Using C
- Programiz – C Programming
- Stack Overflow – For community discussions and troubleshooting
- W3Schools – C Language